A Dual Time-Scale Offloading Strategy for Vehicle Digital Twins via Cloud-Edge-End Collaboration

2025 International Conference on Sensor-Cloud and Edge Computing System (SCECS) | 979-8-3315-2068-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/SCECS65243.2025.11065846

1st Yuqi Wang
College of Computer Science and Technology
Zhejiang University of Technology
Hangzhou, China
221124120174@zjut.edu.cn

2nd Xinggang Fan*
Zhijiang College
Zhejiang University of Technology
Shaoxing, China
xgfan@zjut.edu.cn

3rd Juntao Xu
College of Computer Science and Technology
Zhejiang University of Technology
Hangzhou, China
221124120276@zjut.edu.cn

4th Xiaotong Bi
College of Computer Science and Technology
Zhejiang University of Technology
Hangzhou, China
bixiaotong@zjut.edu.cn

5th Yuzhu Liang*
Advanced Institute of Natural Sciences
Beijing Normal University
Zhuhai, China
cs_yuzhuliang@163.com

***Abstract*—With the advancement of vehicular digital twin (DT) technologies, managing data transmission and task offloading among vehicles, edge servers, and the cloud remains challenging. This paper presents a Double Time-Scale Hierarchical Soft Actor-Critic (DTH-SAC) strategy for collaborative digital twin offloading, addressing the trade-offs among model accuracy, latency, and energy consumption in vehicular networks. The long time scale optimizes generalized model deployment and edge server selection for global stability, while the short time scale adjusts data upload frequency, offloading decisions, and resource allocation for real-time updates. By facilitating edge server collaboration, DTH-SAC effectively mitigates issues from frequent base station handovers, maintaining model accuracy and consistency. Experimental results show that DTH-SAC outperforms SAC and PSO in accuracy, energy efficiency, and latency, providing a flexible solution for intelligent computing in vehicular networks.**

***Index Terms*—Edge Computing, Digital Twin, Internet of Vehicles (IoV), Dual Time-Scale**

I. Introduction

With the rapid development of Digital Twin (DT) technology, its application in the Internet of Vehicles (IoV) has become essential for intelligent transportation systems [1], [2]. Vehicle Digital Twin (VDT) systems enable precise simulation, prediction, and optimization, supporting tasks like autonomous decision-making and fault diagnosis. However, high vehicle mobility and large-scale real-time data processing pose challenges. Therefore, designing a multi-objective optimization strategy for task scheduling and computation offloading is vital for balancing latency, accuracy, and energy consumption, enhancing resource utilization and service quality [3], [4].

Cloud-edge-end collaborative frameworks provide a solution by enabling DTs to operate at the edge, ensuring connectivity, data consistency, and low-latency feedback. While similar approaches have been applied in industrial scenarios [5], VDTs face challenges due to unstable PT-VT connectivity caused by high mobility and rapidly changing vehicle data. This paper addresses how to optimize latency, energy consumption, and accuracy in VDT systems with limited resources using a cloud-edge-end framework to maintain PT-VT connectivity [6], [7].

Although recent studies on DT and edge computing have made progress, some limitations remain. Zhang et al. [8] constructed DTs at the edge without considering edge server switching issues due to PT mobility. Yang et al. [9] designed a Hierarchical Digital Twin (HDT) that considers PT mobility, but it fails to ensure VDT stability under high-speed mobility scenarios. In terms of task scheduling, Al-Habob et al. [10] proposed a genetic algorithm-based task scheduling strategy, which suffered from long inference latency and unstable connections.

To address these challenges, this study proposes a dual time-scale cloud-edge-end collaborative scheduling method for VDT systems in IoV. Unlike traditional methods treating edge nodes as independent units [11]–[13], our approach

*Corresponding author: Xinggang Fan and Yuzhu Liang.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

introduces coordination among edge servers to mitigate PT-VT connectivity issues from high-speed vehicle movement. By monitoring bandwidth fluctuations and varying server loads, we dynamically adjust the offloading strategy in real-time based on network conditions and PT-customized data size. For task scheduling, we optimize latency, energy consumption, and accuracy while configuring data precision and collaboration requirements to ensure reliable PT-VT connectivity. The main contributions of this paper are summarized as follows:

1) We identify and formally define a novel Dual Time-Scale Collaborative Offloading (DTSCO) problem in VDT systems, which arises from the challenge of maintaining system stability and data timeliness under high vehicle mobility.
2) A dual time-scale edge dynamic scheduling algorithm is designed, which fully considers system stability and data timeliness under high-speed vehicle mobility. By monitoring the load status of edge servers in real-time, it flexibly adjusts DT precision and task offloading strategies to achieve multi-objective optimization of latency, energy consumption, and accuracy.
3) Experimental results demonstrate that the proposed approach significantly reduces system latency and energy consumption while maintaining accuracy, benefiting from the dual time-scale design.

The remainder of this paper is organized as follows: Section II describes the system model and problem formulation. Section III details the proposed DTH-SAC algorithm. Section IV evaluates the performance of our algorithm. Finally, Section V concludes the paper.

II. System Modeling and Problem Formulation

A. System Model

This study considers a dual time-scale cloud-edge-end collaborative scheduling problem in an IoV and DT scenario, as illustrated in Fig. 1. The system consists of PTs, Edge Servers (ESs), a cloud server, and communication networks. Let the set of vehicles be denoted by $\mathcal{I}$, with cardinality $|\mathcal{I}| = I$, and the set of spatially distributed ESs be denoted by $\mathcal{M}$, with $|\mathcal{M}| = M$. Each PT generates customized data and transmits it to an ES to form its corresponding VT. The VT construction involves two key components: deployment of a generalized model and integration of customized data. Each PT can form a corresponding VT at the edge, and one ES can host VTs for multiple PTs. The generalized model is downloaded from the cloud to the edge server at a certain precision, while customized data are uploaded from vehicles via onboard sensors. Tasks generated in the VDT system can be executed locally at the PT or offloaded to an ES. If a vehicle moves out of the coverage of the ES where its VT is deployed, the customized data are transmitted to another accessible ES via edge-to-edge collaboration, thereby ensuring data consistency.

Let $\mathcal{T}$ denote the time horizon, which is divided into $T \in \mathbb{N}^+$ time frames, each consisting of $K \in \mathbb{N}^+$ time slots. Let

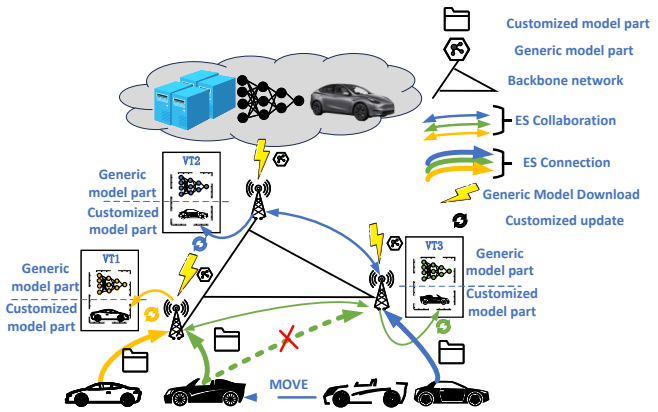


Fig. 1: Cloud-edge-end collaborative process of VDT

$t \in \mathcal{T} = \{0, 1, \ldots, T-1\}$ denote the index of a time frame, and let $\tau \in \mathcal{T}_t = \{tK, tK+1, \ldots, tK+K-1\}$ denote the index of a time slot within time frame t.

B. Communication Model

During time frame $t \in \mathcal{T}$, if VT for vehicle i is deployed on ES m, then $M_{i,m}(t) = 1$. The precision of the generalized model for PT i in time frame t is denoted by $p_i^d(t)$, and the size of the full model is $D_i(t)$. The download latency is:

$$T_i^{\text{down}}(t) = \sum_{m \in M} M_{i,m}(t) \frac{p_i^d(t) D_i(t)}{r^c}, \quad \forall i \in I, \forall t \in \mathcal{T} \quad (1)$$

where r^c is the downlink transmission rate. The corresponding energy consumption is given by $E_i^{\text{down}}(t) = T_i^{\text{down}}(t) \cdot p^c$, where p^c is the ES transmission power.

After downloading, the deployment latency of the generalized model on the ES is:

$$T_{i,m}^{\text{place}}(t) = \frac{M_{i,m}(t) p_i^d(t) D_i(t) C_m}{p_i^f(tK) F_m}, \quad (2)$$

where C_m denotes the number of CPU cycles required per data unit, F_m is the ES's computational capacity, and $p_i^f(tK)$ is the allocated computing resource ratio. The deployment energy consumption $E_{i,m}^{\text{place}}(t)$ is modeled based on the CMOS energy model [12] as:

$$E_{i,m}^{\text{place}}(t) = \rho_m (F_m)^3 \cdot T_{i,m}^{\text{place}}(t) \cdot p_i^f(tK), \quad (3)$$

where ρ_m is the effective switching capacitance of the chip.

The upload latency of customized data is:

$$T_{i,m}^{\text{upload}}(\tau) = \frac{p_i^s(\tau) S_i(\tau)}{\text{rate}_{i,m}(\tau)}, \quad \forall i \in I, \forall \tau \in \mathcal{T}_t, \quad (4)$$

where $p_i^s(\tau)$ is the customized data precision, $S_i(\tau)$ is the data size, and $\text{rate}_{i,m}(\tau)$ is the uplink rate. The corresponding upload energy consumption is $E_{i,m}^{\text{upload}}(\tau) = T_{i,m}^{\text{upload}}(\tau) \cdot p_i^v$, with p_i^v being the PT transmission power.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

If the PT is connected to an ES different from its VT-hosting ES ($m' \neq m$), the cooperative transmission latency between ESs is:

$$T_{i,m}^{\text{col}}(\tau) = \frac{p_i^s(\tau) S_i(\tau)}{r^c} + \sum_{m' \in M} M_{i,m,m'}(\tau) h_{m,m'} t', \quad (5)$$

where $h_{m,m'}$ is the hop count, and t' is the delay per hop. The cooperation energy consumption is $E_{i,m}^{\text{col}}(\tau) = T_{i,m}^{\text{col}}(\tau) \cdot p^c$.

The update latency at the ES after uploading is:

$$T_{i,m}^{\text{update}}(\tau) = \frac{M_{i,m}(t) p_i^s(\tau) S_i(\tau) C_m}{p_i^f(\tau) F_m}, \quad (6)$$

The corresponding update energy consumption is:

$$E_{i,m}^{\text{update}}(\tau) = \rho_m (F_m)^3 \cdot T_{i,m}^{\text{update}}(\tau) \cdot p_i^f(\tau), \quad (7)$$

Once the VT is constructed, the resulting VDT task can either be offloaded to the ES or processed locally. The offloading latency is:

$$T_{i,m}^{\text{offload}}(\tau) = \frac{\lambda_i(\tau)}{\text{rate}_{i,m}(\tau)}, \quad (8)$$

The corresponding energy consumption is:

$$E_{i,m}^{\text{offload}}(\tau) = T_{i,m}^{\text{offload}}(\tau) \cdot p_i^v. \quad (9)$$

If local execution is chosen, the execution latency and energy are:

$$T_i^{\text{exe}}(\tau) = \sum_{m \in M} M_{i,m}(t) O_i(\tau) \frac{\lambda_i(\tau) C_m}{p_i^f(\tau) F_m} + (1 - O_i(\tau)) \frac{\lambda_i(\tau) C_i}{F_i}. \quad (10)$$

$$E_i^{\text{exe}}(\tau) = \sum_{m \in M} M_{i,m}(t) O_i(\tau) \rho_m (F_m)^2 \lambda_i(\tau) C_m + (1 - O_i(\tau)) \rho_i (F_i)^2 \lambda_i(\tau) C_i, \quad (11)$$

where $O_i(\tau) = 1$ indicates offloading, C_i and F_i are the vehicle's computation cycle and frequency.

The precision of the VT is determined by both the generalized and customized data:

$$A_i(\tau) = O_i(\tau) g_i^{\text{ed}}(D_i(\tau), S_i(\tau)) + (1 - O_i(\tau)) g_i^{\text{lo}}, \quad (12)$$

where g_i^{ed} and g_i^{lo} represent edge and local execution precision levels.

C. Problem Formulation

The objective of the DTSCO problem is to jointly optimize the average precision, latency, and energy consumption, formulated as follows:

$$\mathcal{A} = \frac{1}{TK} \sum_{t \in \mathcal{T}} \sum_{\tau \in \mathcal{T}_t} \sum_{i \in \mathcal{I}} A_i(\tau), \quad (13)$$

$$T_{\text{all}} = \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{I}} T_i^{\text{all}}(t), \quad (14)$$

$$E_{\text{all}} = \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{I}} E^{\text{all}}(t). \quad (15)$$

The joint optimization problem is defined as:

$$\min_{\mathcal{J}_i^L(t), \mathcal{J}_i^s(\tau)} \quad \lim_{t \to \infty} (-\mathcal{A},\ T_{\text{all}},\ E_{\text{all}}) \quad (16)$$

$$\text{s.t.} \quad \sum_{m \in \mathcal{M}} M_{i,m}(t) \leq 1, \qquad \forall i \in \mathcal{I},\ \forall t \in \mathcal{T}, \quad (17)$$

$$\sum_{i \in \mathcal{I}} M_{i,m}(t) p_i^b(\tau) \leq 1, \quad \forall m \in \mathcal{M},\ \forall t, \quad (18)$$

$$\sum_{i \in \mathcal{I}} M_{i,m}(t) p_i^f(\tau) \leq 1, \quad \forall m \in \mathcal{M},\ \forall t, \quad (19)$$

$$\frac{1}{K} T_i^{\text{all}}(t) \leq T_i^{\max}, \qquad \forall i \in \mathcal{I},\ \forall t \in \mathcal{T}_t, \quad (20)$$

$$\frac{1}{K} E^{\text{all}}(t) \leq E^{\max}, \qquad \forall i \in \mathcal{I},\ \forall t \in \mathcal{T}_t. \quad (21)$$

where $\mathcal{J}i^L(t) = \{p_i^d(t), Mi, m(t)\}$ denotes the set of large time-scale decisions, and $\mathcal{J}i^s(\tau) = \{p_i^s(\tau), p_i^d(\tau), p_i^f(\tau), Mi, m, m'(\tau), O_i(\tau)\}$ represents the set of small time-scale decisions.

III. Proposed Solution

To address the challenges of multi-task scheduling and multi-objective optimization in VDT systems, we propose a novel dynamic edge task scheduling algorithm based on the SAC framework, named **Double Time-Scale Hierarchical Soft Actor-Critic (DTH-SAC)**, as shown in Algorithm 1. This method integrates scheduling strategies on both large and small time scales, leveraging a hierarchical structure with two levels of policy networks (Actors) and value networks (Critics) to collaboratively optimize multiple objectives.

The state space of SAC is defined as $\{D(t), V_x(tK), V_y(tK), V_v(tK)\}$, where $D(t)$ denotes the set of generalized model sizes for vehicles in the current time frame, $V_x(tK)$ and $V_y(tK)$ represent the x and y coordinates of vehicles at time slot tK, and $V_v(tK)$ denotes the set of vehicle speeds. The high-level action space is $A_h = \{p^d(t), Es_{\text{select}}(t)\}$, where $p^d(t)$ is the set of generalized model precisions, and $Es_{\text{select}}(t)$ represents the selected edge server index for each vehicle. The former is a continuous action space with values in $[0, 1]$, and the latter

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

is a discrete space with values in $\{0, 1, ..., |\mathcal{M}| - 1\}$. The low-level action space is $A_l = \{p^s(\tau), p^f(\tau), C(\tau), O(\tau)\}$, where $p^s(\tau)$ represents customized data precision, $p^f(\tau)$ denotes computing resource allocation ratios, $C(\tau)$ indicates cooperation parameters, and $O(\tau)$ refers to offloading decisions. The first two are continuous spaces with ranges in $[0, 1]$, while the latter two are discrete with values in $\{0, 1\}$.

Algorithm 1 Hierarchical Deep Reinforcement Learning: DTH-SAC

Input: Environment state set $T_{tK} = \{D(t), V_x(tK), V_y(tK), V_v(tK)\}$

Output: High- and low-level actions A_h and A_l

```
Initialize high- and low-level policy networks, Q-value networks, optimizers, and replay buffer
for each episode k from 1 to K do
    Reset environment and obtain initial states (s_h^0, s_l^0)
    for each high-level step from 1 to H do
        Observe high-level state and sample action from high-level policy
        Execute high-level action
        Observe next high-level reward, state, and done signal
        Initialize low-level reward
        for each low-level step from 1 to L do
            Observe low-level state and sample action from low-level policy
            Execute low-level action
            Observe next low-level reward, state, and done signal
            Store (S_low, A_low, R_low, S_next^l, done_l) in replay buffer
            Update low-level SAC network
            Update state
        end for
        Store (S_high, A_high, R_high, S_next^h, done_h) in replay buffer
        Update high-level SAC network
    end for
end for
```

DTH-SAC employs a hierarchical architecture with high-level and low-level decision-making layers. The high-level agent handles macro-level tasks such as model precision and edge server selection, while the low-level agent manages detailed task configurations. This division improves resource allocation efficiency and simplifies policy learning in dynamic, multi-objective environments. DTH-SAC introduces **edge-to-edge cooperation**, ensuring stable and continuous digital twins even when vehicles leave the coverage of their current ES, effectively mitigating VT fluctuations caused by high-speed mobility.

IV. Experiments

A. Experimental Setup

In this experiment, we simulate a vehicular digital twin scenario in the IoV environment, involving task offloading between vehicles and edge servers, computing resource allocation, cooperation, and offloading decisions. The generalized model size of vehicles is randomly selected within $[73.2, 97.6]$ Mbits, the customized data size is within $[6.1, 12.2]$ Mbits, and the VDT task size is within $[10, 20]$ Mbits. The computing frequency F_i of vehicles is set to 1 GHz, while the computing frequency F_m of edge servers is 20 GHz. The downlink transmission rate r^c is 50 Mbps, the downlink transmission power p^c is 5 W, and the vehicle transmission power p_i is 500 mW [14].

Initially, the number of edge servers is set to 4, and each time frame contains $K = 10$ time slots. The number of vehicles is set to 60, 90, and 120, respectively.

B. Baseline Algorithms

To evaluate the performance of the proposed DTH-SAC algorithm, we compare it with the following classical task scheduling algorithms: SAC [15] and PSO [16].

C. Experimental Results and Analysis

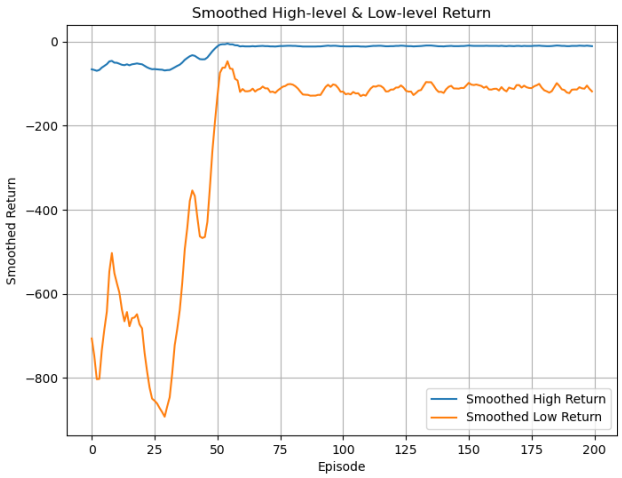
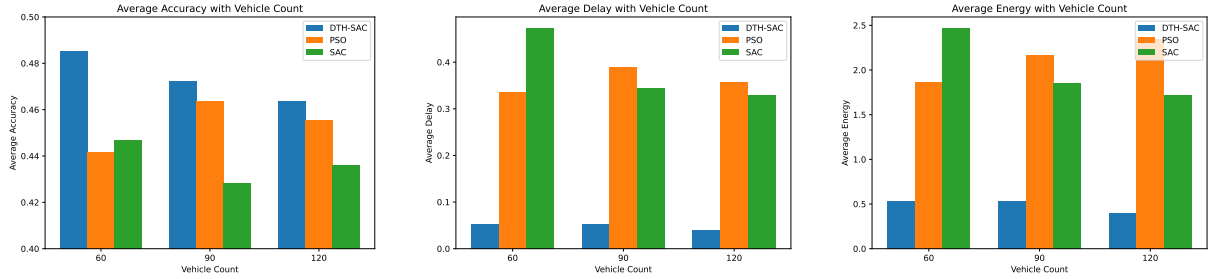


Fig. 2: High-level and low-level returns of DTH-SAC

1) Convergence Analysis: Fig. 2 illustrates the high-level and low-level return curves of DTH-SAC. The high-level return (blue line) steadily increases, indicating effective optimization of the high-level policy. The low-level return (orange line) fluctuates initially but stabilizes over time, reflecting the adaptation and improved performance of the low-level policy as training progresses.

2) Impact on Average Accuracy: Fig. 3a displays the average accuracy across different vehicle numbers. DTH-SAC consistently outperforms other methods, especially at 60 vehicles. PSO shows the worst accuracy, particularly with fewer vehicles, where it is significantly lower than the others. SAC

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.



(a) Average accuracy (b) Average latency (c) Average energy consumption

Fig. 3: Performance comparison of three algorithms under different vehicle numbers: (a) average accuracy, (b) average latency, and (c) average energy consumption.

performs moderately but exhibits fluctuations at 90 and 120 vehicles. As vehicle numbers increase, all methods experience a slight accuracy decline, but DTH-SAC remains the most stable and effective.

3) Impact on Average Latency: Fig. 3b shows the average latency for varying vehicle numbers. DTH-SAC achieves significantly lower latency than PSO and SAC. The higher latency of PSO and SAC results from single time-scale scheduling, which requires frequent model downloads and deployments at each time step. In contrast, DTH-SAC's dual time-scale approach separates high-level resource allocation from low-level task execution, reducing deployment frequency and effectively lowering latency.

4) Impact on Average Energy Consumption: Fig. 3c shows the average energy consumption across different vehicle numbers. DTH-SAC achieves much lower energy consumption compared to PSO and SAC. The higher energy consumption of PSO and SAC is due to frequent model deployments and downloads resulting from single time-scale scheduling.

V. Conclusion

This paper investigates the task scheduling and offloading problem in vehicle digital twin systems within the IoV environment and proposes a dual time-scale dynamic edge task scheduling algorithm, DTH-SAC. By integrating high-level and low-level decision-making, the algorithm effectively addresses the instability between PTs and ESs caused by high-speed vehicle movement. The dual time-scale scheduling strategy balances latency, energy consumption, and accuracy. Extensive experiments demonstrate that DTH-SAC significantly outperforms traditional single time-scale methods in terms of accuracy, delay, and energy consumption, offering an efficient and reliable solution for digital twin scheduling in IoV systems.

References

[1] Y. Liang, M. Yin, Y. Zhang, W. Wang, W. Jia, and T. Wang, "Grouping reduces energy cost in directionally rechargeable wireless vehicular and sensor networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 8, pp. 10 840–10 851, 2023.

[2] S. Sundar, J. P. Champati, and B. Liang, "Multi-user task offloading to heterogeneous processors with communication delay and budget constraints," *IEEE Transactions on Cloud Computing*, vol. 10, no. 3, pp. 1958–1974, 2020.

[3] Y. Liang, M. Yin, W. Wang, Q. Liu, L. Wang, X. Zheng, and T. Wang, "Collaborative edge server placement for maximizing qos with distributed data cleaning," *IEEE Transactions on Services Computing*, pp. 1–15, 2025.

[4] Q. Liu, Y. Peng, J. Wu, T. Wang, and G. Wang, "Secure multi-keyword fuzzy searches with enhanced service quality in cloud computing," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 2046–2062, 2020.

[5] P. Jia, X. Wang, and X. Shen, "Accurate and efficient digital twin construction using concurrent end-to-end synchronization and multi-attribute data resampling," *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 4857–4870, 2023.

[6] Y. Liang, G. Li, G. Zhang, J. Guo, Q. Liu, J. Zheng, and T. Wang, "Latency reduction in immersive systems through request scheduling with digital twin networks in collaborative edge computing," *ACM Transactions on Sensor Networks*, 2024.

[7] X. Liu, P. Lin, T. Liu, T. Wang, A. Liu, and W. Xu, "Objective-variable tour planning for mobile data collection in partitioned sensor networks," *IEEE Transactions on Mobile Computing*, vol. 21, no. 1, pp. 239–251, 2020.

[8] Y. Zhang, H. Zhang, Y. Lu, W. Sun, L. Wei, Y. Zhang, and B. Wang, "Adaptive digital twin placement and transfer in wireless computing power network," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 10 924–10 936, 2024.

[9] Y. Yang, Y. Shi, C. Yi, J. Cai, J. Kang, D. Niyato, and X. Shen, "Dynamic human digital twin deployment at the edge for task execution: A two-timescale accuracy-aware online optimization," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 12 262–12 279, 2024.

[10] A. A. Al-Habob, O. A. Dobre, A. G. Armada, and S. Muhaidat, "Task scheduling for mobile edge computing using genetic algorithm and conflict graphs," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 8, pp. 8805–8819, 2020.

[11] T. Wang, Y. Liang, X. Shen, X. Zheng, A. Mahmood, and Q. Z. Sheng, "Edge computing and sensor-cloud: Overview, solutions, and directions," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–37, 2023.

[12] C. Yi, J. Cai, K. Zhu, and R. Wang, "A queueing game based management framework for fog computing with strategic computing speed control," *IEEE Transactions on Mobile Computing*, vol. 21, no. 5, pp. 1537–1551, 2022.

[13] T. Wang, Y. Liang, Y. Zhang, X. Zheng, M. Arif, J. Wang, and Q. Jin, "An intelligent dynamic offloading from cloud to edge for smart iot systems with big data," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2598–2607, 2020.

[14] T. Wang, Y. Liang, W. Jia, M. Arif, A. Liu, and M. Xie, "Coupling resource management based on fog computing in smart city systems," *Journal of Network and Computer Applications*, vol. 135, pp. 11–19, 2019.

[15] Y. Xie, Q. Wu, and P. Fan, "Digital twin vehicular edge computing network: Task offloading and resource allocation," in *2024 7th International Conference on Information Communication and Signal Processing (ICICSP)*, 2024, pp. 1137–1141.

[16] M. Daviran, A. Maghsoudi, and R. Ghezelbash, "Optimized ai-mpm: Application of pso for tuning the hyperparameters of svm and rf algorithms," *Computers & Geosciences*, vol. 195, p. 105785, 2025.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.